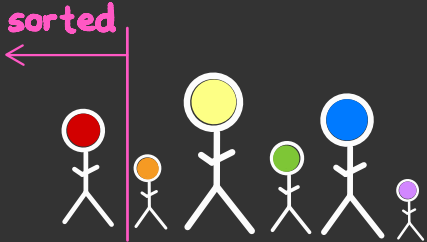


worst-case analysis

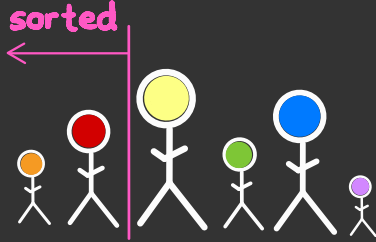
CSCI
134

recall: insertion sort

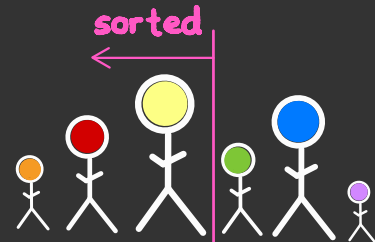
stage 1: ensure the first student is sorted



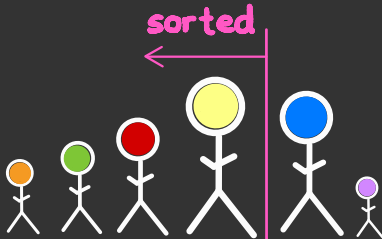
stage 2: ensure the first 2 students are sorted



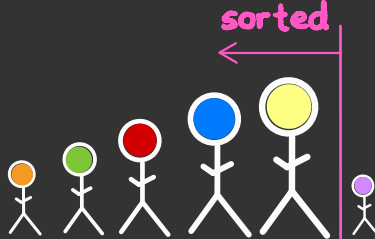
stage 3: ensure the first 3 students are sorted



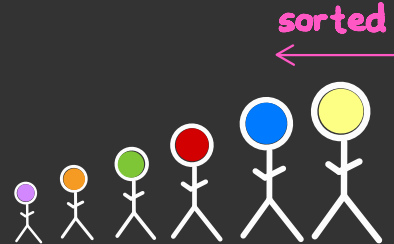
stage 4: ensure the first 4 students are sorted



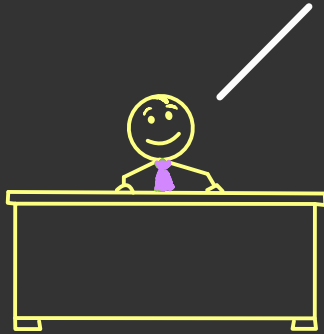
stage 5: ensure the first 5 students are sorted



stage 6: ensure the first 6 students are sorted



how **fast** is ?
insertion sort ?



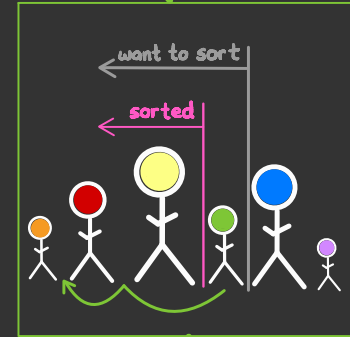
how might we
answer this
question?

to simplify the analysis, let's first decompose the code into two functions

```
def insertion_sort(list):  
    for i in range(len(list)):  
        index = i  
        while index > 0 and list[index-1] > list[index]:  
            list[index], list[index-1] = list[index-1], list[index]  
            index = index - 1
```

```
def insert(index, list):  
    """Keeps moving the element at the specified index left,  
    until it reaches a smaller element."""  
    while index > 0 and list[index-1] > list[index]:  
        list[index], list[index-1] = list[index-1], list[index]  
        index = index - 1
```

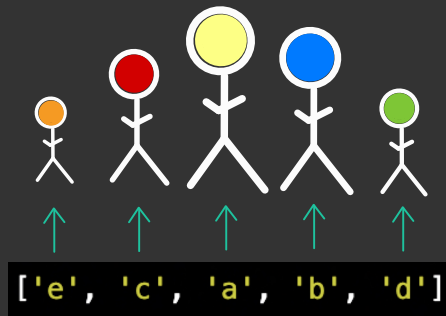
```
def insertion_sort(list):  
    for index in range(len(list)):  
        insert(index, list)
```



if the list has n elements,
insert is called n times

```
def insert(index, list):  
    """Keeps moving the element at the specified index left,  
    until it reaches a smaller element."""  
    while index > 0 and list[index-1] > list[index]:  
        list[index], list[index-1] = list[index-1], list[index]  
        index = index - 1  
  
def insertion_sort(list):  
    for index in range(len(list)):  
        insert(index, list)
```

called n times



```
In [2]: alpha = ['e', 'c', 'a', 'b', 'd']  
  
In [3]: insert(0, alpha)  
  
In [4]: alpha  
Out[4]: ['e', 'c', 'a', 'b', 'd']  
  
In [5]: insert(1, alpha)  
  
In [6]: alpha  
Out[6]: ['c', 'e', 'a', 'b', 'd']  
  
In [7]: insert(2, alpha)  
  
In [8]: alpha  
Out[8]: ['a', 'c', 'e', 'b', 'd']  
  
In [9]: insert(3, alpha)  
  
In [10]: alpha  
Out[10]: ['a', 'b', 'c', 'e', 'd']  
  
In [11]: insert(4, alpha)  
  
In [12]: alpha  
Out[12]: ['a', 'b', 'c', 'd', 'e']
```

if the list has **n** elements,
insert is called **n** times

```
def insert(index, list):  
    """Keeps moving the element at the specified index left,  
    until it reaches a smaller element."""  
    while index > 0 and list[index-1] > list[index]:  
        list[index], list[index-1] = list[index-1], list[index]  
        index = index - 1  
  
def insertion_sort(list):  
    for index in range(len(list)):  
        insert(index, list)
```

called
n times



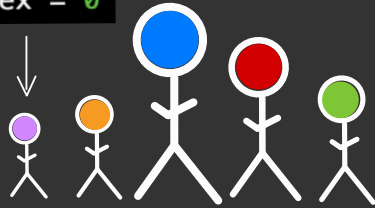
but how **fast** is
each call to **insert**?

```
In [2]: alpha = ['e', 'c', 'a', 'b', 'd']  
  
In [3]: insert(0, alpha)  
  
In [4]: alpha  
Out[4]: ['e', 'c', 'a', 'b', 'd']  
  
In [5]: insert(1, alpha)  
  
In [6]: alpha  
Out[6]: ['c', 'e', 'a', 'b', 'd']  
  
In [7]: insert(2, alpha)  
  
In [8]: alpha  
Out[8]: ['a', 'c', 'e', 'b', 'd']  
  
In [9]: insert(3, alpha)  
  
In [10]: alpha  
Out[10]: ['a', 'b', 'c', 'e', 'd']  
  
In [11]: insert(4, alpha)  
  
In [12]: alpha  
Out[12]: ['a', 'b', 'c', 'd', 'e']
```

sometimes the loop ends **immediately**

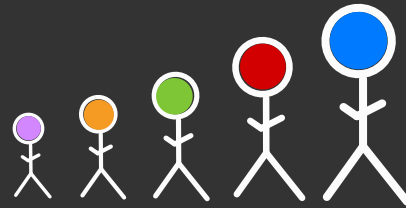
```
def insert(index, list):  
    """Keeps moving the element at the specified index left,  
    until it reaches a smaller element."""  
    while index > 0 and list[index-1] > list[index]:  
        list[index], list[index-1] = list[index-1], list[index]  
        index = index - 1
```

index = 0



list

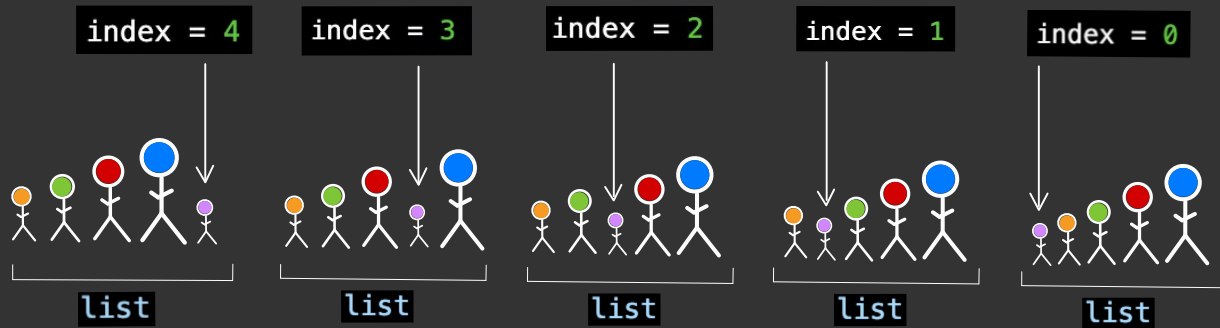
index = 4



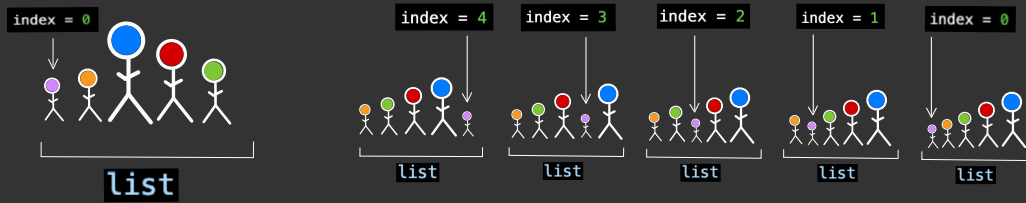
list

but sometimes we loop **n** times

```
def insert(index, list):  
    """Keeps moving the element at the specified index left,  
    until it reaches a smaller element."""  
    while index > 0 and list[index-1] > list[index]:  
        list[index], list[index-1] = list[index-1], list[index]  
        index = index - 1
```

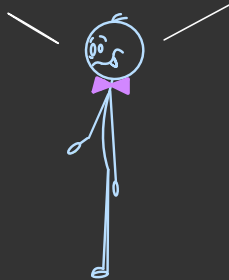


so how do we advertise the speed of insert?



sometimes it
takes 1 step

and sometimes it
takes n steps



in computer
science, we typically
follow the lead of
pharmaceutical
companies

advertise the
worst-case
scenario

"I was taking an antidepressant, but my depression was still creeping up on me."

If you've been on an antidepressant for at least 6 weeks and are still struggling with depression, having ABILIFY added to your antidepressant may help with unresolved symptoms in as early as 1-2 weeks.*

ABILIFY is a prescription medicine used to treat depression in adults as add-on treatment to an antidepressant when an antidepressant alone is not enough.

Important Safety Information

Elderly patients with dementia-related psychosis (eg, an inability to perform daily activities due to increased memory loss) taking ABILIFY have an increased risk of death or stroke. ABILIFY is not approved for treating these patients.

Antidepressants can increase suicidal thoughts and behaviors in children, teens, and young adults. Serious mental illnesses are themselves associated with an increase in the risk of suicide. When taking ABILIFY call your doctor right away if you have new or worsening depression symptoms, unusual changes in behavior, or thoughts of suicide. Patients and their caregivers should be especially observant within the first few months of treatment or after a change in dose. Approved only for adults 18 and over with depression.

- Call your doctor if you develop very high fever, rigid muscles, shaking, confusion, sweating, or increased heart rate and blood pressure, as these may be signs of a rare but potentially fatal condition called neuroleptic malignant syndrome (NMS)
- If you develop uncontrollable facial or body movements, call your doctor, as these may be signs of tardive dyskinesia (TD). TD may become permanent and the risk of TD may increase with the length of treatment and the overall dose. While TD can develop after taking the medicine at low doses for short periods, this is much less common. There is no known treatment for TD, but it may go away partially or completely if the medicine is stopped
- If you have diabetes or have risk factors or symptoms of diabetes, your blood sugar should be monitored. High blood sugar has been reported with ABILIFY and medicines like it. In some cases, extreme high blood sugar can lead to coma or death

• Other risks may include lightheadedness upon standing, decreases in white blood cells (which can be serious), seizures, trouble swallowing, or impairment in judgment or motor skills. Until you know how ABILIFY affects you, you should not drive or operate machinery

The common side effects in adults in clinical trials (≥10%) include nausea, vomiting, constipation, headache, dizziness, an inner sense of restlessness or need to move (akathisia), anxiety, and insomnia. Tell your doctor about all the medicines you're taking, since there are some risks for drug interactions. You should avoid alcohol while taking ABILIFY.

You are encouraged to report negative side effects of prescription drugs to the FDA. Visit www.fda.gov/medwatch or call 1-800-FDA-1088.

Please read the additional Important Information about ABILIFY on the adjacent page.



Ask your doctor about the option of adding ABILIFY.

Register for a free ABILIFY offer* at ABILIFYfreeOffer.com.

*Based on 6-week clinical studies comparing ABILIFY + antidepressant versus antidepressant alone.

†Restrictions apply.



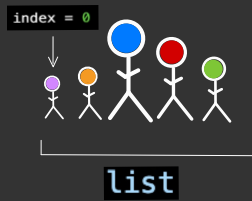
Bristol-Myers Squibb

Otsuka America Pharmaceutical, Inc.

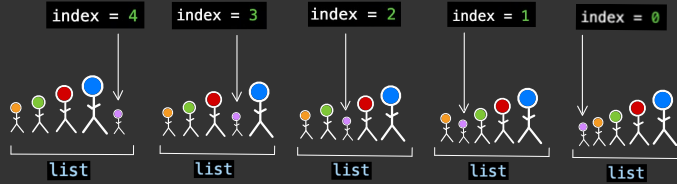
January 2011

02156-1159

so this is how we advertise the speed of insert



~~sometimes it
takes 1 step~~



~~And~~ sometimes it
takes n steps



therefore, if someone were to ask:

how fast is
this code?



```
def linear_search(element, list):  
    for list_element in list:  
        if list_element == element:  
            return True  
    return False
```

we identify the worst-case scenario

element

'q'

(or anything not in `list`)

list

['p','e','s','s','i','m','i','s','m']

A series of eight pink curved arrows pointing from left to right, positioned below the list ['p','e','s','s','i','m','i','s','m'], representing the steps of a linear search through the list.

and determine how many
operations are required

sometimes it
takes **n steps**

